Summarize

Chapter 14. Managing Analytical Data

Tuesday, May 31, 13:23



Logan and Dana (the data architect) were standing outside the big conference room, chatting after the weekly status meeting.

"How are we going to handle analytical data in this new architecture?" asked Dana. "We're splitting the databases into small parts, but we're going to have to glue all that data back together for reporting and analytics. One of the improvements we're trying to implement is better predictive planning, which means we are using more data science and statistics to make more strategic decisions. We now have a team that thinks about analytical data, and we need a part of the system to handle this need. Are we going to have a data warehouse?"

Logan said, "We looked into creating a data warehouse, and while it solved the consolidation problem, it had a bunch of issues for us."

Much of this book has been concerned with how to analyze trade-offs within existing architectural styles such as microservices. However, the techniques we highlight can also be used to understand brand-new capabilities as they appear in the software development ecosystem; *data mesh* is an excellent example.

Analytical and operational data have widely different purposes in modern architectures (see "The Importance of Data in Architecture"); much of this book has dealt with the difficult trade-offs associated with operational data. When client/server systems became popular and powerful enough for large enterprises, architects and database administrators looked for a solution that would allow specialized queries.

Previous Approaches

The split between operational and analytical data is hardly a new problem—the fundamental different uses of data have existed as long as data. As architecture styles have emerged

and evolved, approaches for how to handle data have changed and evolved similarly.

The Data Warehouse

Back in earlier eras of software development (for example, mainframe computers or early personal computers), applications were monolithic, including code and data on the same physical system. Not surprisingly, given the context we've covered up until this point, transaction coordination across different physical systems became challenging. As data requirements became more ambitious, coupled with the advent of local area networks in offices, this led to the rise of *client/server* applications, where a powerful database server runs on the network and desktop applications run on local computers, accessing data over the network. The separation of application and data processing allowed better transactional management, coordination, and numerous other benefits, including the ability to start utilizing historical data for new purposes, such as analytics.

Architects made an early attempt to provide queriable analytical data with the *Data Warehouse* pattern. The basic problem they tried to address goes to the core of the separation of operational and analytical data: the formats and schemas of one don't necessarily fit (or even allow the use of) the other. For example, many analytical problems require aggregations and calculations, which are expensive operations on relational databases, especially those already operating under heavy transactional load.

The Data Warehouse patterns that evolved had slight variations, mostly based on vendor offerings and capabilities. However, the pattern had many common characteristics. The basic assumption was that operational data was stored in relational databases directly accessible via the network. Here are the main characteristics of the Data Warehouse pattern:

Data extracted from many sources

As the operational data resided in individual databases, part of this pattern specified a mechanism for extracting the data into another (massive) data store, the "warehouse" part of the pattern. It wasn't practical to query across all the various databases in the organization to build reports, so the data was extracted into the warehouse solely for analytical purposes.

Transformed to single schema

Often, operational schemas don't match the ones needed for reporting. For example, an operational system needs to structure schemas and behavior around transactions, whereas an analytical system is rarely OLTP data (see Chapter 1) but typically deals with large amounts of data, for reporting, aggregations, and so on. Thus, most data warehouses utilized a *Star Schema* to implement dimensional modelling, transforming data from operational systems in differing formats into the warehouse schema.

To facilitate speed and simplicity, warehouse designers denormalize the data to facilitate performance and simpler queries.

Loaded into warehouse

Because the operational data resides in individual systems, the warehouse must build mechanisms to regularly extract the data, transform it, and place it in the warehouse. Designers either used built-in relational database mechanisms like replication or specialized tools to build translators from the original schema to the warehouse schema. Of course, any changes to operational systems schemas must be replicated in the transformed schema, making change coordination difficult.

Analysis done on the warehouse

Because the data "lives" in the warehouse, all analysis is done there. This is desirable from an operational standpoint: the data warehouse machinery typically featured massively capable storage and compute, offloading the heavy requirements into its own ecosystem.

Used by data analysts

The data warehouse utilized data analysts, whose job included building reports and other business intelligence assets. However, building useful reports requires domain understanding, meaning that domain expertise must reside in both the operational data system and the analytical systems, where query designers must use the same data in a transformed schema to build meaningful reports and business intelligence.

BI reports and dashboards

The output of the data warehouse included business intelligence reports, dashboards that provide analytical data, reports, and any other information to allow the company to make better decisions.

SQL-ish interface

To make it easier for DBAs to use, most data warehouse query tools provided familiar affordances, such as a SQL-like language for forming queries. One of the reasons for the data transformation step mentioned previously was to provide users with a simpler way to query complex aggregations and other intelligence.

THE STAR SCHEMA

The *Star Schema pattern* was popular with data marts and warehouses. It separates the data semantics into *facts*, which hold the organization's quantifiable data, and *dimensions*; hence they are also known as *dimensional models*, which include descriptive attributes of the fact data.

Examples of fact data for the Sysops Squad might include hourly rate, time to repair, distance to client, and other concretely measurable things. Dimensions might include squad member specialties, squad person names, store locations, and other metadata.

Most significantly, the Star Schema is purposely denormalized to facilitate simpler queries, simplified business logic (in other words, fewer complex joins), faster queries and aggregations, complex analytics such as data cubes, and the ability to form multidimensional queries. Most Star Schemas become incredibly complex.

The Data Warehouse pattern provides a good example of *technical partitioning* in software architecture: warehouse designers transform the data into a schema that facilitates queries and analysis but loses any domain partitioning, which must be re-created in queries where required. Thus, highly trained specialists were required to understand how to construct queries in this architecture.

However, the major failings of the Data Warehouse pattern included integration brittleness, extreme partitioning of domain knowledge, complexity, and limited functionality for intended purpose:

Integration brittleness

The requirement built into this pattern to transform the data during the injection phase creates crippling brittleness in systems. A database schema for a particular problem domain is highly coupled to the semantics of that problem; changes to the domain require schema changes, which in turn require data import logic changes.

Extreme partitioning of domain knowledge

Building complex business workflows requires domain knowledge. Building complex reports and business intelligence also requires domain knowledge, coupled with specialized analytics techniques. Thus, the Venn diagrams of domain expertise overlap, but only partially. Architects, developers, DBAs, and data scientists must all coordinate on data changes and evolution, forcing tight coupling between vastly different parts of the ecosystem.

Complexity

Building an alternate schema to allow advanced analytics adds complexity to the system, along with the ongoing mechanisms required to injest and transform data. A data warehouse is a separate project outside the normal operational systems for an organization, so must be maintained as a wholly separate ecosystem, yet highly coupled to the domains embedded inside the operational systems. All these factors contribute to complexity.

Limited functionality for intended purpose

Ultimately, most data warehouses failed because they didn't deliver business value commensurate to the effort required to create and maintain the warehouse. Because this pattern was common long before cloud environments, the physical investment in infrastructure was huge, along with the ongoing development and maintenance. Often, data consumers would request a certain type of report that the warehouse couldn't provide. Thus, such an ongoing investment for ultimately limited functionality doomed most of these projects.

Synchronization creates bottlenecks

The need in a data warehouse to synchronize data across a wide variety of operational systems creates both operational and organizational bottlenecks—a location where multiple and otherwise independent data streams must converge. A common side effect of the data warehouse is the synchronization process impacting operational systems despite the desire for decoupling.

Operational versus analytical contract differences

Systems of record have specific contract needs (discussed in Chapter 13). Analytical systems also have contractual needs that often differ from the operational ones. In a data warehouse, the pipelines often handle the transformation as well as ingestion, introducing contractual brittleness in the transformation process.

Table 14-1 shows the trade-offs for the data warehouse pattern.

TRADE-OFFS

Table 14-1. Trade-offs for the Data Warehouse pattern

Advantage	Disadvantage
Centralized consolidation of data	Extreme partitioning of domain knowledge
Dedicated analytics silo provides isolation	Integration brittleness
	Complexity
	Limited functionality for intended purpose

Tuesday, May 31, 13:33



"We looked at creating a data warehouse, but realized that it fit better with older, monolithic kinds of architectures than modern distributed ones," said Logan. "Plus, we have a ton more machine learning cases now that we need to support."

"What about the data lake idea I've been hearing about?" asked Dana. "I read a blog post on Martin Fowler's site.[1] It seems like it addresses a bunch of the issues with the data warehouse, and it is more suitable for ML use cases."

"Oh, yes, I read that post when it came out," Logan said. "His site is a treasure trove of good information, and that post came out right after the topic of microservices became hot. In fact, I first read about microservices on that same site in 2014, and one of the big questions at the time was, *How do we manage reporting in architectures like that?* The data lake was one of the early answers, mostly as a counter to the data warehouse, which definitely won't work in something like microservices."

"Why not?" Dana asked.

The Data Lake

As in many reactionary responses to the complexity, expense, and failures of the data warehouse, the design pendulum swung to the opposite pole, exemplified by the *Data Lake* pat-

tern, intentionally the inverse of the Data Warehouse pattern. While it keeps the centralized model and pipelines, it inverts the "transform and load" model of the data warehouse to a "load and transform" one. Rather than do the immense work of transformation, the philosophy of the Data Lake pattern holds that, rather than do useless transformations that may never be used, do no transformations, allowing business users access to analytical data in its natural format, which typically required transformation and massaging for their purpose. Thus, the burden of work was made *reactive* rather than *proactive*—rather than do work that might not be needed, do transformation work only on demand.

The basic observation that many architects made was that the prebuilt schemas in data warehouses were frequently not suited to the type of report or inquiry required by users, requiring extra work to understand the warehouse schema enough to craft a solution. Additionally, many machine learning models work better with data "closer" to the semi-raw format rather than a transformed version. For domain experts who already understood the domain, this presented an excruciating ordeal, where data was stripped of domain separation and context to be transformed into the data warehouse, only to require domain knowledge to craft queries that weren't natural fits of the new schema!

Characteristics of the Data Lake pattern are as follows:

Data extracted from many sources

Operational data is still extracted in this pattern, but less transformation into another schema takes place—rather, the data is often stored in its "raw," or native, form. Some transformation may still occur in this pattern. For example, an upstream system might dump formatted files into a lake that are organized based on a column-based snapshots.

Loaded into the lake

The lake, often deployed in cloud environments, consists of regular data dumps from the operational systems.

Used by data scientists

Data scientists and other consumers of analytical data discover the data in the lake and perform whatever aggregations, compositions, and other transformations necessary to answer specific questions.

The Data Lake pattern, while an improvement in many ways to the Data Warehouse pattern, still suffered many limitations.

This pattern still takes a *centralized* view of data, where data is extracted from operational systems' databases and replicated into a more or less free-form lake. The burden was on

the consumer to discover how to connect disparate data sets together, which often happened in the data warehouse despite the level of planning. The logic followed that, if we're going to have to do pre-work for some analytics, let's do it for all, and skip the massive upfront investment.

While the Data Lake pattern avoided the transformation-induced problems from the Data Warehouse pattern, it also either didn't address or created new problems.

Difficulty in discovery of proper assets

Much of the understanding of data relationships within a domain evaporates as data flows into the unstructured lake. Thus, domain experts must still involve themselves in crafting analysis.

PII and other sensitive data

Concern around PII has risen in concert with the capabilities of the data scientist to take disparate pieces of information and learn privacy-invading knowledge. Many countries now restrict not just private information, but also information that can be combined to learn and identify, for ad targeting or other less savory purposes. Dumping unstructured data into a lake often risks exposing information that can be stitched together to violate privacy. Unfortunately, just as in the discovery process, domain experts have the knowledge necessary to avoid accidental exposures, forcing them to reanalyze data in the lake.

Still technically, not domain, partitioned

The current trend in software architecture shifts focus from partitioning a system based on technical capabilities into ones based on domains, whereas both the Data Warehouse and Data Lake patterns focus on technical partitioning. Generally, architects design each of those solutions with distinct ingestion, transformation, loading, and serving partitions, each focused on a technical capability. Modern architecture patterns favor domain partitioning, encapsulating technical implementation details. For example, the microservices architecture attempts to separate services by domain rather than technical capabilities, encapsulating domain knowledge, including data, inside the service boundary. However, both the Data Warehouse and Data Lake patterns try to separate data as a separate entity, losing or obscuring important domain perspectives (such as PII data) in the process.

The last point is critical—increasingly, architects design around *domain* rather than *technical* partitioning in architecture, and both previous approaches exemplify separating data from its context. What architects and data scientists need is a technique that preserves the ap-

propriate kind of macro-level partitioning, yet supports a clean separation of analytical from operational data. Table 14-2 lists the trade-offs for the Data Lake pattern.

TRADE-OFFS

Table 14-2. Trade-offs for the Data Lake pattern

Advantage	Disadvantage
Less structured than data warehouse	Sometimes difficult to understand relationships
Less up-front transformation	Requires ad hoc transformations
Better suited to distributed architectures	

The disadvantages around brittleness and pathological coupling of pipelines remain. Although they do less transformation in the Data Lake pattern, it is still common, as well as data cleansing.

The Data Lake pattern pushes data integrity testing, data quality, and other quality issues to downstream lake pipelines, which can create some of the same operational bottlenecks that manifest in the Data Warehouse pattern.

Because of both technical partitioning and the batch-like nature, solutions may suffer from data staleness. Without careful coordination, architects either ignore the changes in upstream systems, resulting in stale data, or allow the coupled pipelines to break.

Tuesday, May 31, 14:43



"OK, so we can't use the data lake either!" exclaimed Dana. "What now?"

"Fortunately, some recent research has found a way to solve the problem of analytical data with distributed architectures like microservices," replied Logan. "It adheres to the domain boundaries we're trying to achieve, but also allows us to project analytical data in a way that the data scientists can use. And, it eliminates the PII problems our lawyers are worried about."

"Great!" Dana replied. "How does it work?"

The Data Mesh

Observing other trends in distributed architectures, Zhamak Dehghani and several other innovators derived the core idea of the Data Mesh pattern from domain-oriented decoupling of microservices, service mesh, and sidecars (see "Sidecars and Service Mesh"), and applied it to analytical data, with modifications. As we mentioned in Chapter 8, the *Sidecar Pattern* provides a nonentangling way to organize orthogonal coupling (see "Orthogonal Coupling"); the separation between operational and analytical data is another excellent example of just such a coupling, but with more complexity than simple operational coupling.

Definition of Data Mesh

Data mesh is a sociotechnical approach to sharing, accessing, and managing analytical data in a decentralized fashion. It satisfies a wide range of analytical use cases, such as reporting, ML model training, and generating insights. Contrary to the previous architecture, it does so by aligning the architecture and ownership of the data with the business domains and enabling a peer-to-peer consumption of data.

Data mesh is founded on four principles:

Domain ownership of data

Data is owned and shared by the domains that are most intimately familiar with the data: the domains that either are originating the data, or are the first-class consumers of the data. This architecture allows for distributed sharing and accessing the data from multiple domains and in a peer-to-peer fashion without any intermediary and centralized lake or warehouse, and without a dedicated data team.

Data as a product

To prevent siloing of data and encourage domains to share their data, data mesh introduces the concept of data served as a product. It puts in place the organizational roles and success metrics necessary to ensure that domains provide their data in a way that delights the experience of data consumers across the organization. This principle leads to the introduction of a new architectural quantum called *data product quantum*, to maintain and serve discoverable, understandable, timely, secure, and high-quality data to the consumers. This chapter introduces the architectural aspect of the data product quantum.

Self-serve data platform

To empower the domain teams to build and maintain their data products, data mesh introduces a new set of self-serve platform capabilities. The capabilities focus on im-

proving the experience of data product developers and consumers. It includes features such as declarative creation of data products, discoverability of data products across the mesh through search and browsing, and managing the emergence of other intelligent graphs, such as lineage of data and knowledge graphs.

Computational federated governance

This principle assures that despite decentralized ownership of the data, organization-wide governance requirements—such as compliance, security, privacy, and quality of data, as well as interoperability of data products—are met consistently across all domains. Data mesh introduces a federated decision-making model composed of domain data product owners. The policies they formulate are automated and embedded as code in each and every data product. The architectural implication of this approach to governance is a platform-supplied embedded sidecar in each data product quantum to store and execute the policies at the point of access: data read or write.

Data mesh is a wide-ranging topic, fully covered in the book *Data Mesh* by Zhamak Dehghani (O'Reilly). In this chapter, we focus on the core architectural element, the *data product quantum*.

Data Product Quantum

The core tenet of the data mesh overlays modern distributed architectures such as microservices. Just as in the *service mesh*, teams build a *data product quantum* (DPQ) adjacent but coupled to their service, as illustrated in Figure 14-1.

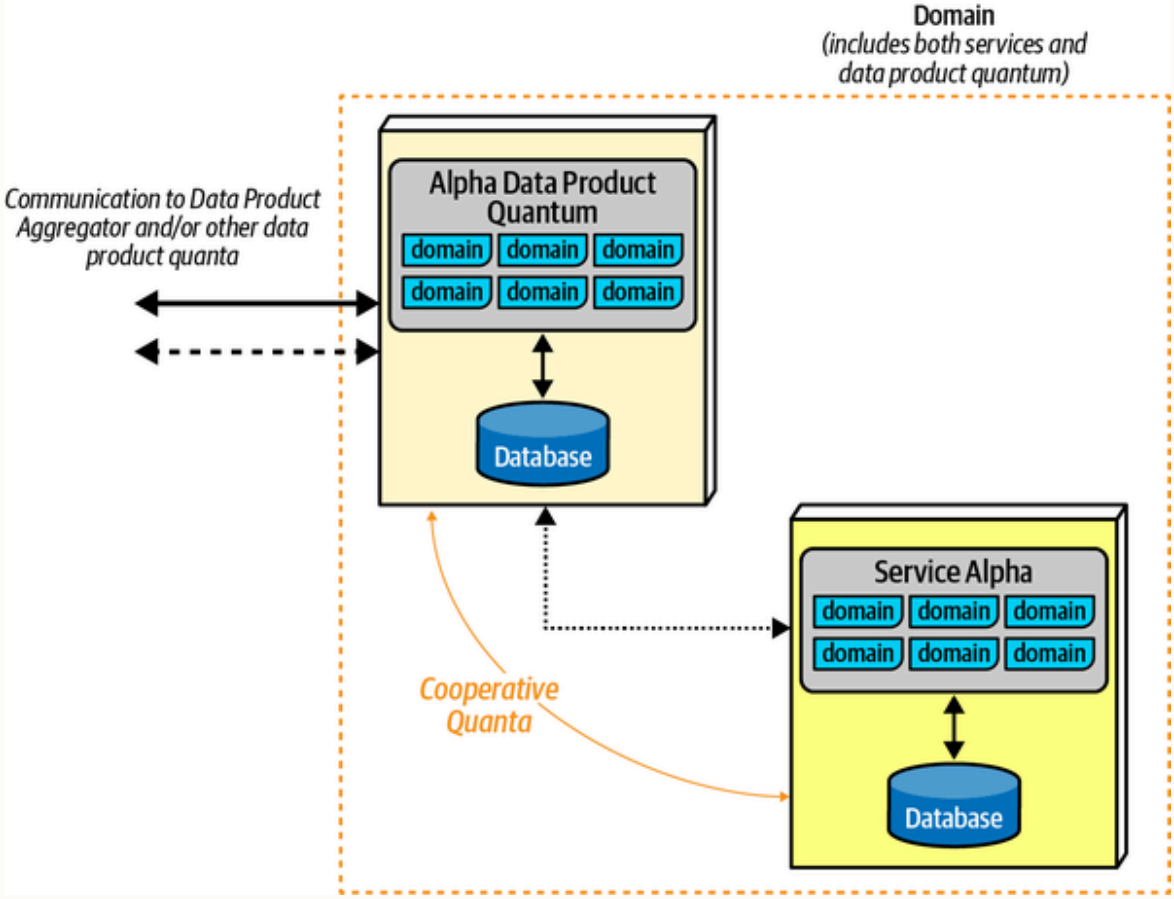


Figure 14-1. Structure of a data product quantum

In this example, the service *Alpha* contains both behavior and transactional (operational) data. The domain includes a *data product quantum*, which also contains code and data, and which acts as an interface to the overall analytical and reporting portion of the system. The DPQ acts as an operationally independent but highly coupled set of behaviors and data.

Several types of DPQs commonly exist in modern architectures:

Source-aligned (native) DPQ

Provides analytical data on behalf of the collaborating architecture quantum, typically a microservice, acting as a cooperative quantum.

Aggregate DQP

Aggregates data from multiple inputs, either synchronously or asynchronously. For example, for some aggregations, an asynchronous request may be sufficient; for others, the aggregator DPQ may need to perform synchronous queries for a source-aligned DPQ.

Fit-for-purpose DPQ

A custom-made DPQ to serve a particular requirement, which may encompass analytical reporting, business intelligence, machine learning, or other supporting capability.

Each domain that also contributes to analysis and business intelligence includes a DPQ, as illustrated in Figure 14-2.

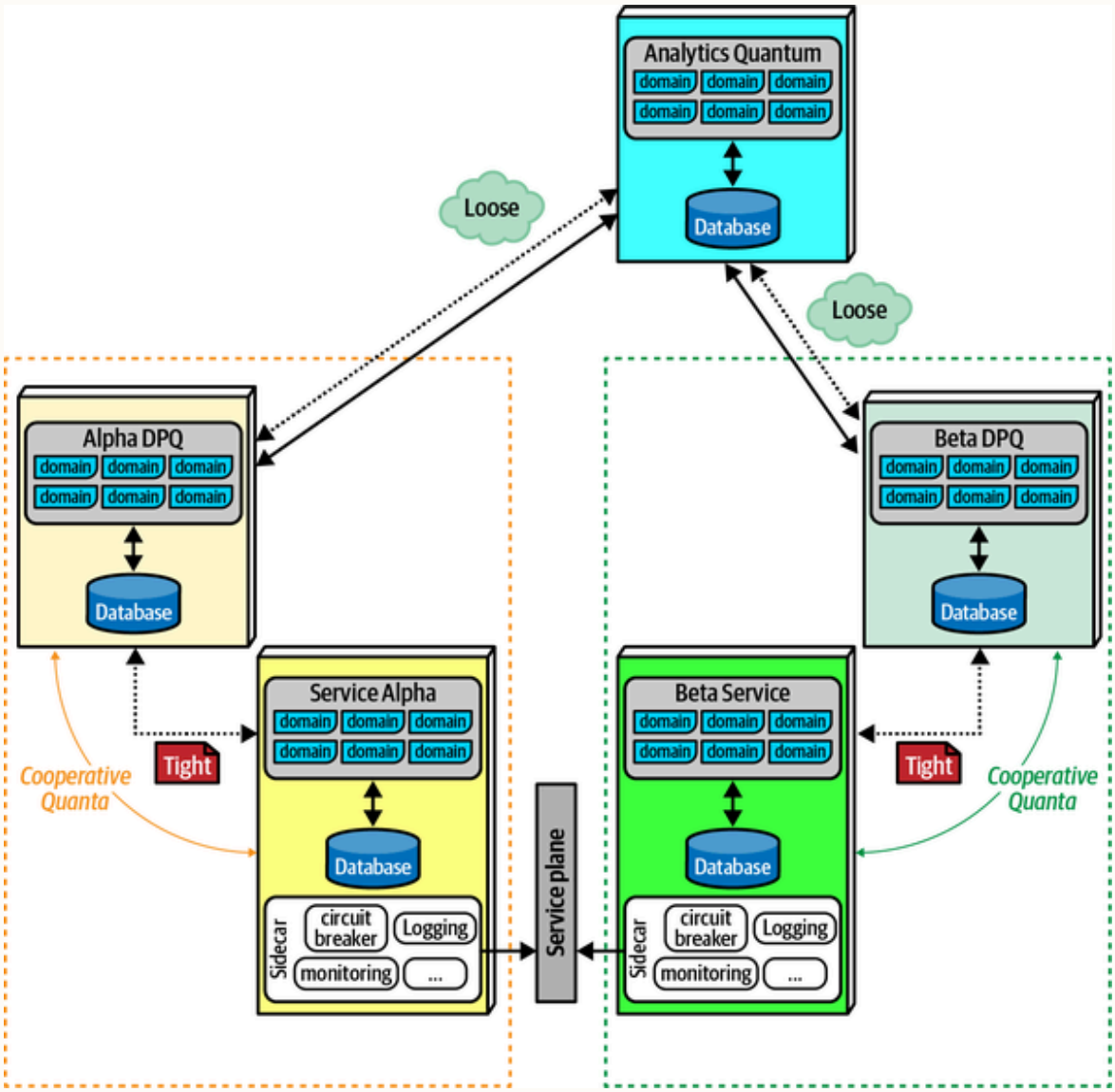


Figure 14-2. The data product quantum acts as a separate but highly coupled adjunct to a service

Here, the DPQ represents a component owned by the domain team responsible for implementing the service. It overlaps information stored in the database, and may have interactions with some of the domain behavior asynchronously. The data product quantum also likely has behavior as well as data for the purposes of analytics and business intelligence.

Each data product quantum acts as a cooperative quantum for the service itself:

Cooperative quantum

> An operationally separate quantum that communicates with its cooperator via asynchronous communication and eventual consistency, yet features tight contract coupling with its cooperator and generally looser contract coupling to the analytics quantum, the service responsible for reports, analysis, business intelligence, and so on. While the two cooperating quanta are operationally independent, they represent

two sides of data: operational data in the quantum and analytical data in the data product quantum.

Some portion of the system will carry the responsibility for analytics and business intelligence, which will form its own domain and quantum. To operate, this analytical quantum has static quantum coupling to the individual data product quanta it needs for information. This service may make either synchronous or asynchronous calls to the DPQ, depending on the type of request. For example, some DPQs will feature a SQL interface to the analytical DPQ, allowing synchronous queries. Other requirements may aggregate information across multiple DPQs.

Data Mesh, Coupling, and Architecture Quantum

Because analytical reporting is probably a required feature of a solution, the DPQ and its communication implementation belong to the static coupling of an architecture quantum. For example, in a microservices architecture, the service plane must be available, just as a message broker must be available if the design calls for messaging. However, like the Sidecar pattern in a service mesh, the DPQ should be orthogonal to implementation changes within the service, and maintain a separate contract with the data plane.

From a dynamic quantum coupling standpoint, the data sidecar should always implement one of the communication patterns that features both eventual consistency and asynchronicity: either the "Parallel Saga$^{(aeo)}$ Pattern" or "Anthology Saga$^{(aec)}$ Pattern". In other words, a data sidecar should never include a transactional requirement to keep operational and analytical data in sync, which would defeat the purpose of using a DPQ for orthogonal decoupling. Similarly, communication to the data plane should generally be asynchronous, so as to have minimal impact on the operational architecture characteristics of the domain service.

When to Use Data Mesh

Like all things in architecture, this pattern has trade-offs associated with it, as shown in Table 14-3.

TRADE-OFFS

Table 14-3. Trade-offs for the Data Mesh pattern

Advantage	Disadvantage
Highly suitable for microservices architectures	Requires contract coordination with data product quantum
Follows modern architecture principles and engineering practices	Requires asynchronous communication and eventual consistency
Allows excellent decoupling between analytical and operational data	
Carefully formed contracts allow loosely coupled evolution of analytical capabilities	

It is most suitable in modern distributed architectures such as microservices with well-contained transactionality and good isolation between services. It allows domain teams to determine the amount, cadence, quality, and transparency of the data consumed by other quanta.

It is more difficult in architectures where analytical and operational data must stay in sync at all times, which presents a daunting challenge in distributed architectures. Finding ways to support eventual consistency, perhaps with very strict contracts, allows many patterns that don't impose other difficulties.

Data mesh is an outstanding example of the constant incremental evolution that occurs in the software development ecosystem; new capabilities create new perspectives, which in turn help address some persistent headaches from the past, such as the artificial separation of domain from data, both operational and analytical.

Sysops Squad Saga: Data Mesh

Friday, June 10, 09:55

Logan, Dana, and Addison met in the big conference room, which often had leftover snacks (or, this early in the day, breakfast) from previous meetings.



"I just returned from a meeting with our data scientists, and they are trying to figure out a way we can solve a long-term problem for us—we need to become data-driven in expert supply planning, for skill sets demand for different geographical locations at different points in time. That capability will help recruitment, training, and other supply-related functions," said Logan.

"I haven't been involved in much of the data mesh implementation—how far along are we?" asked Addison.

"Each new service we've implemented includes a DPQ. The domain team is responsible for running and maintaining the DQP cooperative quantum for their service. We've only just started. We're gradually building out the capabilities as we identify the needs. I have a picture of the Ticket Management Domain in Figure 14-3."

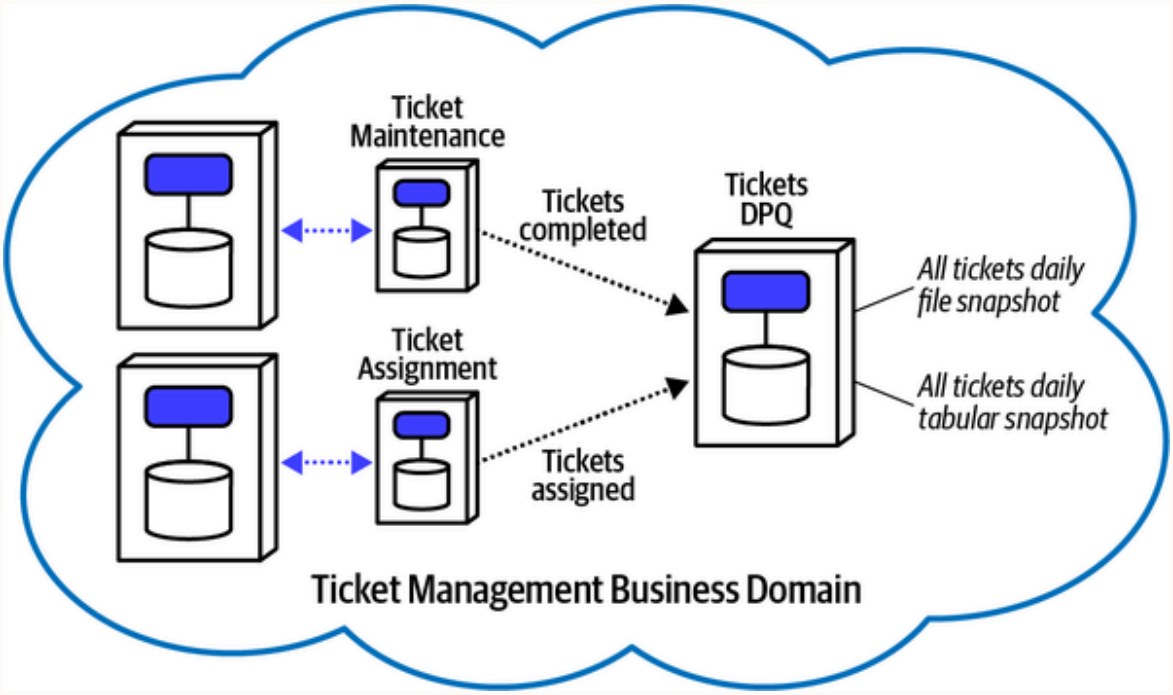


Figure 14-3. Ticket Management Domain, including two services with their own DPQs, with a Tickets DPQ

Logan said, "Tickets DPQ is its own architecture quantum, and acts as an aggregation point for a couple of different ticket views that other systems care about."

"How much does each team have to build versus already supplied?" Addison asked.

"I can answer that," said Dana. "The data mesh platform team is supplying the data users and data product developers with a set of self-serve capabilities. That allows any team that wants to build a new analytical use case to search and find the data products of choice within existing architecture quanta, directly connect to them, and start using them. The platform also supports domains that want to create new data products. The platform continuously monitors the mesh for any data product downtimes, or incompatibility with the governance policies and informs the domain teams to take actions."

Logan said, "The domain data product owners in collaboration with security, legal, risk, and compliance SMEs, as well as the platform product owners, have formed a global federated governance group, which decides on aspects of the DPQs that must be standardized, such as their data-sharing contracts, modes of asynchronous transport of data, access control, and so on. The platform team, over a span of time, enriches the DPQ's sidecar with new policy execution capabilities and upgrades the sidecars uniformly across the mesh."

"Wow, we're further along that I thought," said Dana. "What data do we need in order to supply the information for the expert supply problem?"

Logan replied, "In collaboration with the data scientists, we have determined what information we need to aggregate. It looks like we have the correct information: the Tickets DPQ serves the long-term view of all tickets raised and resolved, the User Maintenance DPQ provides daily snapshots for all expert profiles, and the Survey DPQ provides a log of all survey results from customers."

"Awesome," said Addison. "Perhaps we should create a new DPQ named something like Experts Supply DPQ, which takes asynchronous inputs from those three DPQs? Its first product can be called *supply recommendations*, which uses an ML model trained using data aggregated from DPQs in surveys, tickets, and maintenance domains. The Experts Supply DPQ will provide daily recommendations data, as new data becomes available about tickets, surveys and expert profiles. The overall design looks like Figure 14-4."

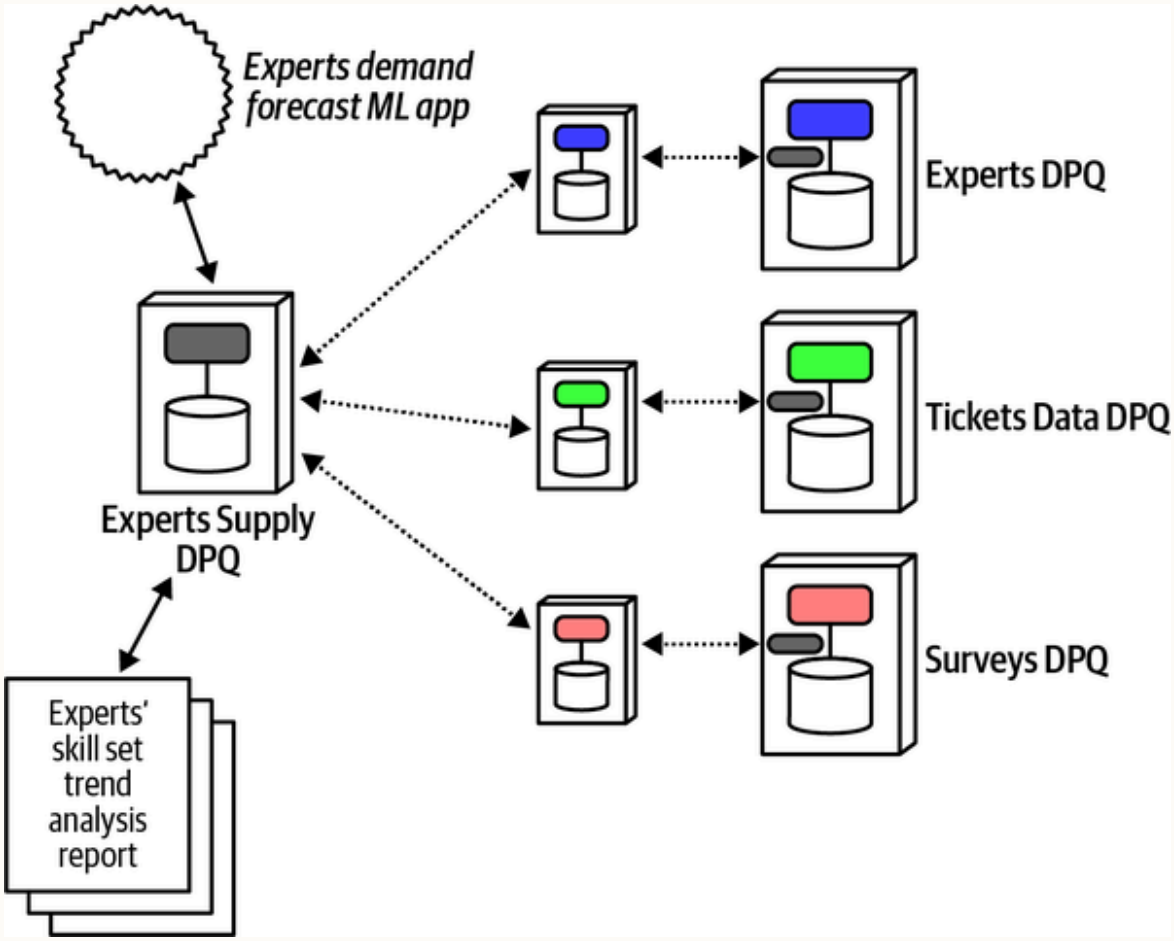


Figure 14-4. Implementing the Experts Supply DPQ

"OK, that looks perfectly reasonable," said Dana. "The services are already done; we just have to make sure the specific endpoints exist in each of the source DPQs, and implement the new Experts Supply DPQ."

"That's right," said Logan. "One thing we need to worry about, though—trend analysis depends on reliable data. What happens if one of the feeder source systems returns incomplete information for a chunk of time? Won't that throw off the trend analysis?"

"That's correct—no data for a time period is better than incomplete data, which makes it seem like there was less traffic than there was," Dana said. "We can just exempt an empty day, as long as it doesn't happen much."

"OK, Addison, you know what than means, right?" Logan said.

"Yes, I certainly do—an ADR that specifies complete information or none, and a fitness function to make sure we get complete data."

> *ADR: Ensure that Expert Supply DPQ Sources Supply an Entire Day's Data or None*
>
> *Context*
> *The Expert Supply DPQ performs trend analysis over specified time periods. Incomplete data for a particular day will skew trend results and should be avoided.*
>
> *Decision*
> *We will ensure that each data source for the Expert Supply DPQ receives complete snapshots for daily trends or no data for that day, allowing data scientists to exempt that day.*
>
> *The contracts between source feeds and the Expert Supply DPQ should be loosely coupled to prevent brittleness.*
>
> *Consequences*
> *If too many days become exempt because of availability or other problems, accuracy of trends will be negatively impacted.*
>
> *Fitness functions:*
>
> *Complete daily snapshot. Check timestamps on messages as they arrive. Given typical message volume, any gap of more than one minute indicates a gap in processing, marking that day as exempt.*
>
> *Consumer-driven contract fitness function for Ticket DPQ and Expert Supply DPQ. To ensure that internal evolution of the Ticket Domain doesn't break the Experts Supply DPQ.*

1 Martin Fowler posted an influential message about the Data Lake pattern on his blog in 2015 at *https://martinfowler.com/bliki/DataLake.html*.

Previous chapter
< 13. Contracts

Next chapter
15. Build Your Own Trade-Off Analysis >